

# PA02 보고서 초안: 고속 상관 스캔 매처 고속화

Cartographer Fast Correlative Scan Matcher | Jetson Nano | CUDA 10.2 | ROS Melodic

## 1. 실험 목적과 조건

PA01에서는 `score_all.cpp`를 대상으로 CPU profiling과 GPU profiling을 수행하였다. 그 결과 CPU에서는 `grid[y * w + x]`에 대한 불규칙한 memory access가 주요 병목으로 확인되었고, 반복 연산 최소화과 OpenMP 병렬화를 적용하더라도 성능 향상은 제한적이었다. 반면 GPU 구현에서는 candidate와 scan point 계산의 독립성을 활용하여 workload가 충분히 커질 때 CPU 대비 큰 성능 향상을 얻을 수 있었다. 그러나 PA01의 GPU 최적화는 주로 `score_all()` 단일 함수와 kernel 내부 reduction 방식에 초점을 맞추었기 때문에, scan별 CUDA 반복 호출, 매 호출마다 발생하는 memory allocation/copy/free, 그리고 상위 scoring path의 host-side overhead는 여전히 남아 있었다.

따라서 PA02에서는 PA01의 결론을 출발점으로 삼아, `MakeLowCands()` → `FastMatcher::Score()` → `score_all()`로 이어지는 coarse scoring path 전체를 대상으로, scan별로 분리되던 CUDA score 호출을 여러 scan을 하나의 kernel launch로 묶는 방식, 즉 batch CUDA 구조로 재구성하였다. 이는 `score_all()` kernel 단독 최적화가 아니라, CUDA 반복 호출 overhead와 kernel 내부의 memory access 비효율을 동시에 줄이는 접근이다.

Fast Correlative Scan Matcher는 branch-and-bound 탐색의 초기 단계에서 저해상도 precomputed grid 기준으로 다수의 후보 pose(coarse candidate)를 생성하고, 각 candidate 위치에서 scan point들이 map grid와 얼마나 잘 맞는지 score를 계산한다. 이 단계에서 상위 score를 받은 소수 candidate만 이후 branch refinement에서 정밀 재평가된다. `score_all()`은 candidate 위치마다 scan point를 map grid 위에 이동시킨 뒤, 각 point가 참조하는 grid 값을 모두 더해 candidate score를 계산한다. candidate 수를  $N$ , scan point 수를  $P$ 라고 하면 한 scan에 대한 score 계산량은  $O(NP)$ 이고, 여러 scan을 처리하면 전체 계산량은 scan 수에 비례하여 증가한다. 즉 candidate 수와 scan point 수가 증가할수록 scoring path는 전체 scan matching의 핵심 병목이 된다.

이번 과제에서는 Jetson Nano에서 측정한 결과만 사용하였다. 최적화 효과의 확장성을 확인하기 위해 grid resolution과 탐색 depth를 변경하여 workload를 60.3M → 237M → 1.47B → 5.68B로 단계적으로 늘려가며 분석하였다.

### 1.1 사용한 profiling 도구

본 실험에서는 병목을 score 연산 수준, CUDA 호출 수준, CUDA kernel metric 수준으로 나누어 분석하였다. 먼저 C++ 코드 내부에는 `std::chrono` 기반 timer를 삽입하여 `MatchWithWindow()`, `MakeScans()`, `MakeLowCands()`, `FastMatcher::Score()`, `Branch()`의 평균 실행 시간을 측정하였다. 이를 통해 전체 matcher 실행 시간 중 어떤 함수가 가장 큰 비중을 차지하는지 확인하였다.

CUDA score 호출 내부 시간은 CUDA event로 측정하였다. H2D copy, CUDA kernel, D2H copy, host-side overhead를 분리하여 측정함으로써 batch CUDA 호출에서 병목이 데이터 전송인지 kernel 실행인지 구분하였다.

CUDA runtime과 kernel 수준의 상세 분석에는 NVIDIA `nvprof`를 사용하였다. `--print-gpu-trace`와 `--print-api-trace`로 `ScoreAllKernel`, H2D/D2H copy, `cudaMalloc`, `cudaMemcpyAsync`, `cudaFree`, `cudaLaunchKernel`, `cudaStreamSynchronize`의 호출 수와 누적 시간을 확인하였다. 또한 `--metrics`로 achieved occupancy, SM efficiency, global load/store efficiency, warp execution efficiency, branch efficiency, IPC를 수집하여 kernel 내부 병목을 분석하였다.

| 도구 / 방법                               | 측정 대상                                              | 사용 목적                     |
|---------------------------------------|----------------------------------------------------|---------------------------|
| <code>std::chrono</code> timer        | matcher 내부 함수별 실행 시간                               | 전체 흐름에서 병목 함수 확인          |
| CUDA event                            | H2D, kernel, D2H, host-side overhead               | CUDA score 호출 내부 시간 분해    |
| <code>nvprof --print-gpu-trace</code> | kernel, H2D/D2H copy                               | GPU activity 기준 병목 확인     |
| <code>nvprof --print-api-trace</code> | malloc, memcpy, free, launch, sync                 | CUDA runtime API 호출 비용 확인 |
| <code>nvprof --metrics</code>         | occupancy, memory efficiency, warp efficiency, IPC | kernel 내부 병목 원인 분석        |

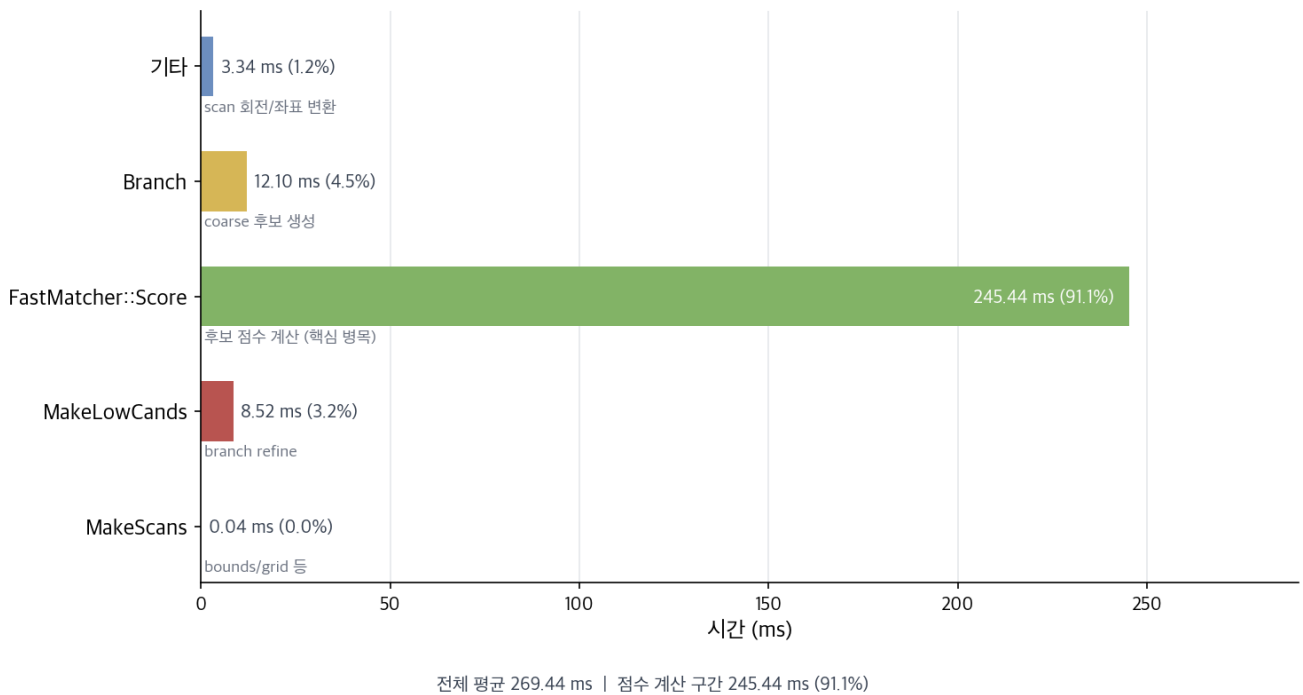
모든 측정은 ROS launch와 rosbag 실행 환경(Jetson Nano, Ubuntu 18.04 / CUDA 10.2)에서 수행하였으며, CUDA context 초기화 warm-up을 제외한 안정 구간의 평균값을 사용하였다. `nvprof`가 profiling하는 CUDA API 총 시간은 rosbag 전체 구간의 누적합이므로 `std::chrono`로 측정한 함수별 평균과 직접 비교할 수 없었기에, CUDA API 항목 간 상대적 비용과 호출 구조의 병목 원인을 해석하는 데만 활용하였다. 안정 구간 내 평균값은 전 조건에서  $\pm 2\%$  이내로 수렴하였으나, workload가 커질수록 호출 간 편차가 증가하는 경향을 보였다(60.3M:  $\pm 6\%$ , 237M 이상:  $\pm 10\%$  이상).

## 2. 기준 CUDA의 상세 profiling

본 보고서에서 기준 CUDA는 기존 CPU 기반 `score_all()`을 candidate 단위로 병렬화한 단순 CUDA 버전이다. candidate 하나를 CUDA block 하나에 대응시키고, block 내 thread들이 scan point loop를 분담하여 처리한 뒤 reduction으로 candidate score를 합산한다. 그러나 scan별로 `score_all()`을 반복 호출하며 매 호출마다 `cudaMalloc`, H2D copy, kernel launch, D2H copy, synchronization, `cudaFree`가 반복되고, `grid[y * width + x]` 접근은 candidate offset에 따라 불규칙하게 달라져 global memory access 효율이 낮다. 이후 profiling에서는 이 기준 CUDA를 baseline으로 삼아, 병목이 kernel 내부 계산과 CUDA 호출 구조 중 어디에서 발생하는지 분석하였다.

### 2.1 함수 수준 병목

기준 CUDA 함수별 실행 시간  
조건: res0.05/depth2, 총 score 연산량 60.3M 회

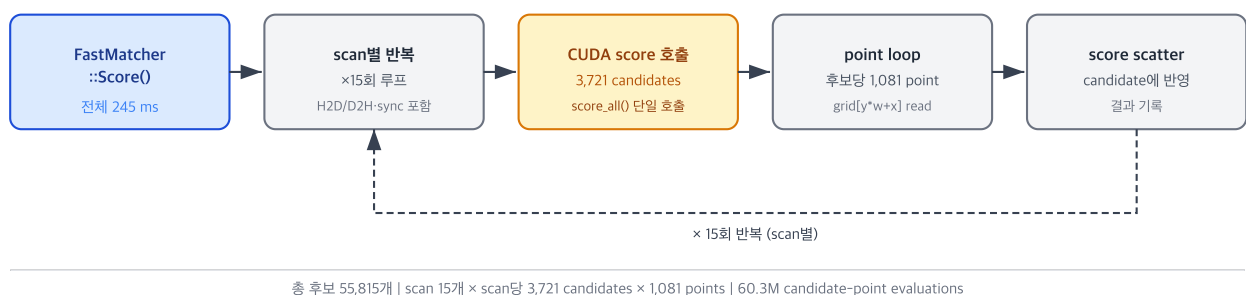


함수별 profiling 결과, 기준 CUDA의 실행 시간은 `FastMatcher::Score()`에 집중되어 있었다. `FastMatcher::Score()`는 `MatchWithWindow()` 전체 실행 시간의 91.1%를 차지한 반면, scan 생성, candidate 생성, branch refinement 단계의 비중은 모두 작았다. 따라서 최적화 대상은 전체 matcher가 아니라 `MakeLowCands()` → `FastMatcher::Score()` → `score_all()`로 이어지는 coarse candidate에 대한 score을 연산하는 함수들로 선정하였다.

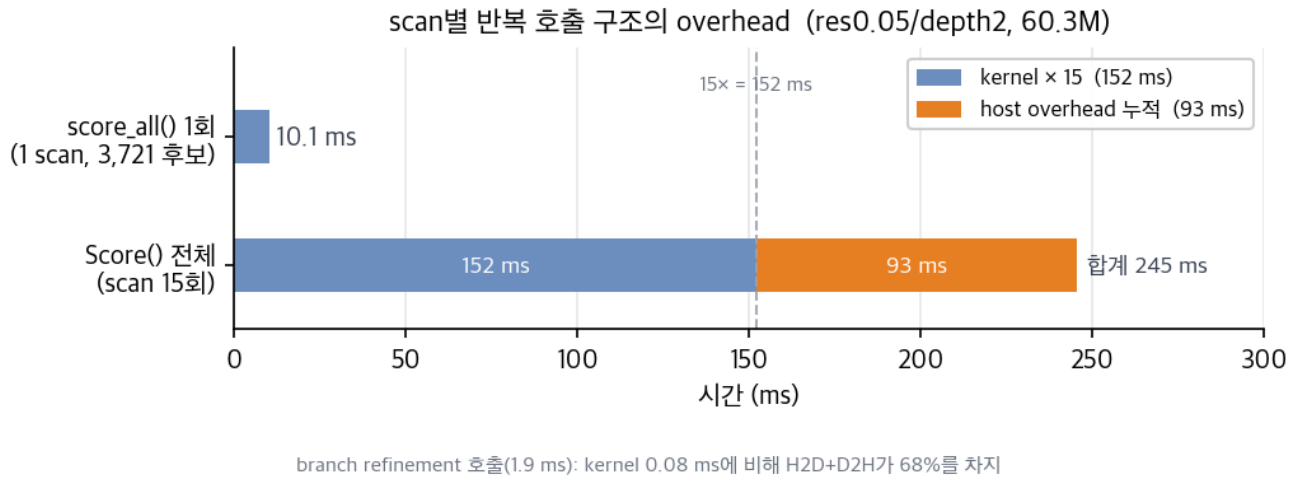
## 2.2 CUDA 호출 수준 병목

기준 CUDA는 coarse candidate를 scan별로 나누어 처리한다. 전체 candidate는 55,815개이지만, 실제 CUDA score 호출은 scan 15개에 대해 각각 3,721개 candidate를 처리하는 구조이다. 즉 60.3M 연산량을 한 번에 처리하지 않고 scan별 CUDA 호출 15회로 나누어 실행한다.

기준 CUDA 구현의 점수 계산 호출 구조



`FastMatcher::Score()`는 scan마다 CUDA 호출을 반복하는 구조로, kernel score 연산 외에도 입력 배열 구성, score scatter, H2D/D2H copy, synchronization 대기가 매 scan마다 반복되어 오버헤드를 야기하였다.



coarse `score_all()` 1회는 약 10 ms이지만 `FastMatcher::Score()` 전체는 245 ms로, 15 scan을 단순 곱한 150 ms보다 약 95 ms가 더 크다. 이 차이는 매 scan마다 반복되는 입력 배열 구성, score scatter, H2D/D2H copy, synchronization 대기 등 host-side 비용이 kernel 시간 위에 누적된 결과이다. branch refinement 호출(1.9 ms)에서는 kernel이 0.08 ms에 불과한 반면 H2D+D2H가 약 68%를 차지하여, 후보 수가 적어 kernel 시간이 짧아도 데이터 전송 비용은 줄지 않음을 보여주었다.

### 2.3 CUDA runtime 수준 상세 병목

res0.05/depth2(60.3M) 조건을 기준으로 nvprof로 측정한 CUDA runtime API 누적 시간을 분석하였다. 총 시간은 rosbag 구간 전체의 누적합이므로 항목 간 상대적 비율로 해석하였다.

| host-side CUDA API 항목              | 총 시간      | 호출 수   | 평균 시간     | 해석                                     |
|------------------------------------|-----------|--------|-----------|----------------------------------------|
| <code>cudaStreamSynchronize</code> | 38.74 s   | 13,954 | 2.78 ms   | kernel 완료 대기 시간이 대부분 포함되었다.            |
| <code>cudaMalloc</code>            | 7.30 s    | 41,862 | 174.32 us | kernel 1회당 6번 반복되어 host-side 비용을 만들었다. |
| <code>cudaMemcpyAsync</code>       | 3.89 s    | 41,862 | 92.97 us  | 작은 copy가 매우 많이 반복되었다.                  |
| <code>cudaFree</code>              | 2.59 s    | 41,862 | 61.76 us  | allocation과 마찬가지로 반복 호출 비용이 컸다.        |
| <code>cudaLaunchKernel</code>      | 705.25 ms | 6,977  | 101.08 us | 작은 branch 호출에서는 launch 비용이 상대적으로 커졌다.  |

`cudaMalloc/cudaFree`가 kernel 1회당 6번씩 발생하고 `cudaStreamSynchronize`가 누적 시간 1위를 차지한 것은, scan별 반복 호출 구조가 `FastMatcher::Score()` 전체 시간을 kernel 시간의 단순 합 이상으로 키우는 직접 원인임을 보여주었다.

device-side에서는 GPU activity의 대부분이 `ScoreAllKernel`이었으며, H2D/D2H copy의 비중은 상대적으로 작았다. kernel 내부 비효율을 구체적으로 파악하기 위해 metric을 추가로 수집하였다.

| CUDA metric               | 평균      | 해석                                                                                                                                                                                       |
|---------------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| achieved occupancy        | 0.36    | SM 자원의 동시 활용 수준이 낮았다. SM efficiency가 높은 점을 함께 고려하면, SM이 idle한 것이 아니라 memory latency 대기로 인해 warp가 충분히 채워지지 않은 것으로 해석되었다.                                                                  |
| SM efficiency             | 99.19%  | SM이 거의 항상 active 상태였다. global load efficiency와 IPC가 낮은 점을 고려하면 SM이 연산보다 memory 응답 대기 시간을 소비한 것으로 볼 수 있었다.                                                                                |
| global load efficiency    | 6.67%   | 같은 block 내 thread들은 동일한 candidate offset(cx, cy)을 공유하지만, 각 thread가 더하는 scan point(px, py)가 달라 <code>grid[y*w+x]</code> 읽기 주소가 분산되었다. 분산된 주소는 하나의 memory transaction으로 묶이지 못해 효율이 극히 낮았다. |
| global store efficiency   | 73.24%  | candidate score를 기록하는 쓰기 연산은 상대적으로 효율이 높았다. 읽기 연산(grid 접근)에서 효율 손실이 집중되었다.                                                                                                               |
| warp execution efficiency | 55.82%  | bounds check에 따른 invalid point 처리, global memory latency로 인한 warp stall, reduction 단계의 lane 비활성화가 복합적으로 작용한 결과였다.                                                                        |
| branch efficiency         | 100.00% | branch divergence는 병목 원인이 아니었다.                                                                                                                                                          |
| IPC                       | 0.27    | clock당 실행 instruction 수가 낮았다. kernel이 global memory 응답을 기다리느라 실제 연산이 지연되었음을 직접 나타냈다.                                                                                                     |

`score_all()` kernel은 연산 능력 부족이 아니라 memory 응답 대기로 비어 있는 시간이 긴 것으로 확인되었다 (global load efficiency 6.67%, IPC 0.27). 이는 shared memory staging과 warp-level reduction으로 kernel 내부 비효율을 줄일 수 있다는 근거가 되었다.

종합하면, 병목은 두 함수에 나뉘어 있었다. `FastMatcher::Score()`는 scan별 반복 호출 구조 자체가 문제였고, `score_all()` kernel은 내부 실행 효율이 낮았다. 두 병목은 원인이 달라 각각 별도로 개선해야 했다.

### 3. 최적화 대상과 선정 이유

2절에서 확인된 두 병목 축—`score_all()` kernel 내부 비효율과 `FastMatcher::Score()`의 scan별 반복 호출 overhead—은 원인이 달랐기 때문에, 각 축을 단독으로 개선한 경우와 둘을 함께 해소한 경우의 효과를 비교하기 위해 단독 개선 4가지와 통합 개선 2가지를 설계하였다. device-side 단독으로는 warp reduction(kernel 변경), shared memory tiling + warp reduction(shmem만), bounds check 최적화(bounds만) 세 가지를 각각 적용하였고, host-side 단독으로는 per-scan 호출 구조를 유지하면서 `cudaMalloc/cudaFree` 반복만 제거하는 buffer 재사용을 시도하였다. 통합 개선에서는 **batch kernel**을 도입하였다. 15개 scan의 point와 candidate를 하나의 flat array로 통합하여 1회 kernel launch로 전체 scan을 처리하는 구조로, scan마다 반복되던 `cudaMalloc/copy/sync` overhead를 한 번에 제거한다. batch kernel에 device-side 기법을 모두 결합한 것이 scan batch이고, 여기에 bounds check 최적화를 추가한 것이 bounds 감소이다. batch kernel로 전환하면 candidate 배열 형태도 달라지므로, `MakeLowCands()`도 수정 대상에 포함하였다.

| 대상 함수 | 선정 이유 | 6가지 구현에서 변경한 내용 |
|-------|-------|-----------------|
|-------|-------|-----------------|

| 대상 함수                                    | 선택 이유                                                                          | 6가지 구현에서 변경한 내용                                                                                                                                          |
|------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>score_all()</code>                 | 핵심 연산부이며 kernel 내부 memory 대기가 주 병목이었다(global load efficiency 6.67%, IPC 0.27). | memory 대기를 줄이기 위해 warp-level reduction, shared memory staging, bounds check 단축을 단계적으로 적용하였다. batch 구현에서는 여러 scan을 하나의 kernel launch로 묶어 처리하도록 구조를 변경하였다. |
| <code>FastMatcher::Score()</code>        | 전체 실행 시간의 91.1%를 차지하였으며, scan별 반복 호출 구조가 병목의 핵심이었다.                            | 먼저 scan마다 반복되던 buffer 할당·해제를 미리 할당해 재사용하는 방식으로 바꾸었고, 최종적으로 15번의 개별 호출을 1번의 batch 호출로 통합하였다.                                                              |
| <code>FastMatcher::MakeLowCands()</code> | 실행 시간 1.5%로 병목은 아니었지만, batch kernel이 요구하는 형태의 입력을 만들기 위해 포함하였다.                | scan별로 나뉘어 있던 candidate를 하나의 연속 배열로 재구성하고, 각 scan의 시작 위치를 offset으로 기록하도록 변경하였다.                                                                          |

## 4. 구현 상세

6가지 구현은 device-side(kernel 내부)와 host-side(호출 구조)를 각각 단독으로 또는 함께 변경하도록 설계하였다. 아래 표는 각 버전이 어떤 축을 변경하는지 요약한다.

| 버전         | warp reduction | shmem tiling | scan batch (1회 호출) | buffer 재사용 | bounds 감소 |
|------------|----------------|--------------|--------------------|------------|-----------|
| 기준 CUDA    |                |              |                    |            |           |
| kernel 변경  | ✓              |              |                    |            |           |
| buffer 재사용 |                |              |                    | ✓          |           |
| shmem만     | ✓              | ✓            |                    |            |           |
| bounds만    |                |              |                    |            | ✓         |
| scan batch | ✓              | ✓            | ✓                  | ✓          |           |
| bounds 감소  | ✓              | ✓            | ✓                  | ✓          | ✓         |

기준 CUDA. candidate 1개 = CUDA block 1개. scan마다 개별 호출, 매 호출마다 `cudaMalloc` → `H2D` → `kernel` → `D2H` → `sync` → `cudaFree`.

**kernel 변경.** block-level reduction을 warp-level `__shfl_down_sync` reduction으로 교체, warp당 candidate 1개로 재구성. 호출 구조·buffer 관리는 *기존 CUDA*와 동일.

**buffer 재사용.** `FastMatcher::Score()` 진입 시 score·candidate용 device buffer를 한 번만 할당하고 이후 scan별 호출에서 재사용. grid가 바뀌지 않으면 H2D grid upload도 생략. kernel 내부는 *기존 CUDA*와 동일.

**shmem만.** kernel 내부에 ① warp-level reduction(kernel 변경과 동일), ② scan point를 256개 단위로 shared memory에 미리 올려두고 처리하는 shared memory tiling 적용. 호출 구조·buffer 관리는 *기존 CUDA*와 동일.

**bounds만.** map 경계 안에 완전히 들어오는 candidate(full\_inside)는 point별 2D 좌표 변환과 범위 초과 여부 확인을 생략하고, 미리 계산해둔 1D `cell_offset[p]`로 grid를 직접 접근. 호출 구조·reduction·buffer 관리는 *기존 CUDA*와 동일.

**scan batch.** 15개 scan의 point와 candidate를 scan 경계 없이 이어붙인 단일 배열로 통합하여 CUDA 호출 15회 → 1회. kernel은 `blockIdx.y`로 scan을 구분하고 `blockIdx.x`로 candidate를 병렬 처리. kernel 내부에 warp reduction + shared memory tiling 포함. scan별 launch overhead와 sync 14회분이 제거됨.

**bounds 감소.** scan batch + bounds만의 bounds check 최적화를 결합한 최종 버전.

## 5. 결과와 비교 분석

세 가지 실험을 통해 다음 질문에 답하였다.

1. **(5.1)** 두 병목을 함께 해소한 버전과 단독 개선 버전의 speedup이 얼마나 다른가?
2. **(5.2)** kernel 효율만 높이면 `FastMatcher::Score()` 전체 시간도 줄어드는가?
3. **(5.3)** 반복 호출 overhead를 제거하고 나면 `Score()` 호출 내부에서 무엇이 지배적 병목이 되는가?

### 5.1 전체 실행 시간 비교

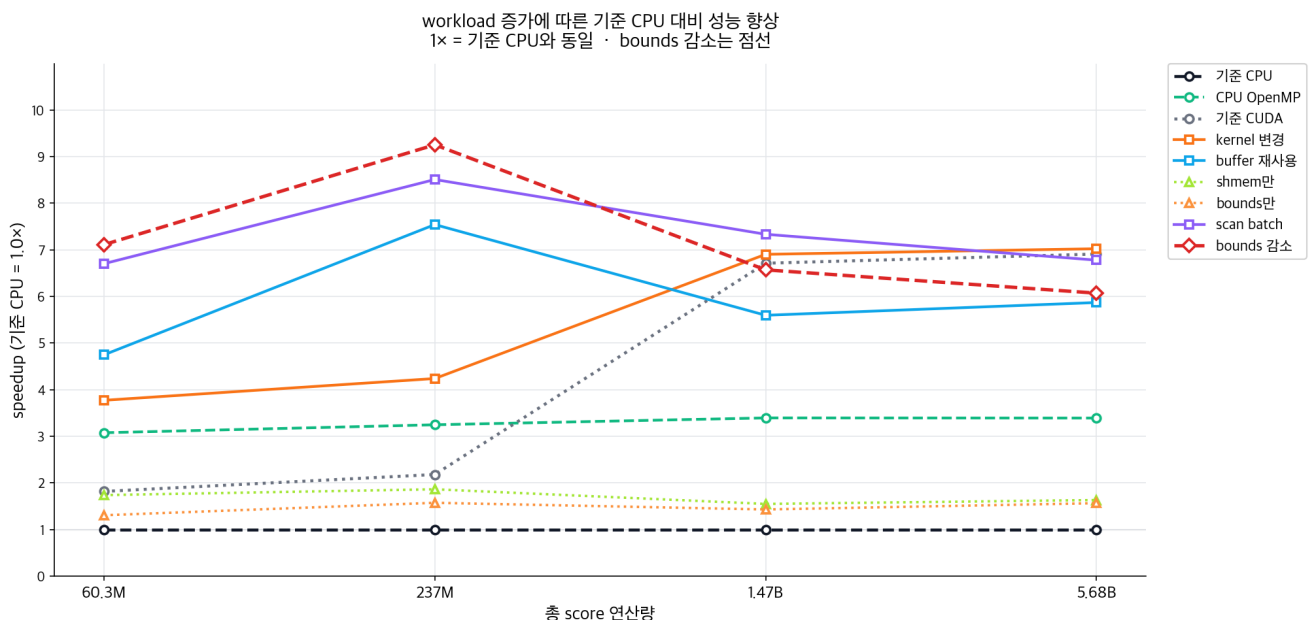


그림 5.1은 각 최적화 버전의 workload별 `MatchWithWindow()` 실행 시간을 기준 CPU 대비 배율(기준 CPU = 1x)로 나타낸다. CPU OpenMP SIMD(4 threads)는 비교 기준으로 추가하였다.

**scan batch·bounds 감소가 60.3M·237M에서 두드러지는 이유.** 연산량이 작은 구간일수록 전체 Score() 시간에서 반복 호출 overhead가 차지하는 비중이 크다. 60.3M에서 kernel 계산 시간을 15배 합한 값이 약 150 ms인데 실제 Score() 시간은 245 ms로, 95 ms가 순수 overhead였다. scan batch는 이 overhead를 없애고 kernel 최적화까지 더했기 때문에 60.3M 기준에서 효과가 가장 두드러졌다. 237M에서 피크를 찍는 것은, workload가 커지면 한 번의 batch 호출로 처리하는 연산량이 늘어 GPU가 더 잘 활용되면서 효율이 극대화되는 구간이기 때문이다. bounds 감소는 여기에 per-point bounds check 생략까지 더해 같은 구간에서 scan batch보다도 빠른 결과를 냈다.

**1.47B에서 기준 CUDA와 kernel 변경만 계속 향상되는 이유.** 이 두 버전은 추가적인 host-side 준비 작업 없이 kernel 을 그대로 실행한다. workload가 커질수록 candidate 수가 늘어 GPU의 SM이 더 많은 block으로 채워지고 GPU 활용 률이 높아지기 때문에, kernel 계산 효율이 개선된다. 특히 기준 CUDA는 60.3M에서는 GPU가 충분히 채워지지 않아 CPU OpenMP(3.08x)에 뒤지지만, 1.47B에서 SM이 포화 수준에 가까워지며 6.71x까지 급격히 향상된다. buffer 재 사용·shmem만·bounds만 은 추가 host 작업 비용이 overhead로 남아 있어 이 효율 상승이 상쇄되는 반면, kernel 변 경은 warp reduction으로 처음부터 처리량이 높아 전 구간에서 기준 CUDA보다 안정적으로 앞섰다.

**5.68B에서 CUDA 버전들이 6~7x 수준으로 모이는 이유.** workload가 매우 커지면 `score_all()`의 불규칙한 `grid[y*w+x]` 접근(global load efficiency 6.67%)에 의한 memory bandwidth 병목이 모든 버전에서 지배적이 된다. 이 시점에서는 호출 구조나 buffer 관리 방식의 차이가 전체 시간에 비해 작아지고, GPU가 달성할 수 있는 실질적 인 처리율이 memory bandwidth 한계로 수렴하기 때문이다. scan batch가 5.68B에서도 상대적으로 하락폭이 적은 것은 호출 overhead가 없어 순수 kernel 시간이 기준이 되고, 그 kernel 자체는 workload가 커져도 GPU를 최대한 활 용하기 때문이다.

**shmem만·bounds만이 CPU OpenMP보다도 저조한 이유.** 두 버전은 kernel 내부를 최적화하면서 per-scan 호출 구조를 그대로 유지하였다. kernel이 빨라진 대신 각 scan마다 host에서 tiling용 배열 구성이나 precomputed offset 배열 준비 같은 추가 작업이 반복된다. 이 작업이 15개 scan에 걸쳐 누적되어 dispatcher 비용이 기준 CUDA 대비 3~4 배 급증하였고, GPU에서 절약한 시간을 CPU에서 소비하는 구조가 되었다. GPU kernel이 빨라질수록 전체 시간에서 이 host 준비 비용의 비중이 더 커지기 때문에, kernel 최적화가 오히려 역효과를 낸 것이다. 자세한 시간 분해는 5.2에서 확 인할 수 있다.

결국 그래프 전체를 관통하는 패턴은 하나다. GPU를 더 잘 쓰는 것과 CPU에서 덜 반복하는 것, 두 가지를 함께 해결했을 때만 의미 있는 개선이 나왔다.

## 5.2 FastMatcher::Score() 내부 시간 분해

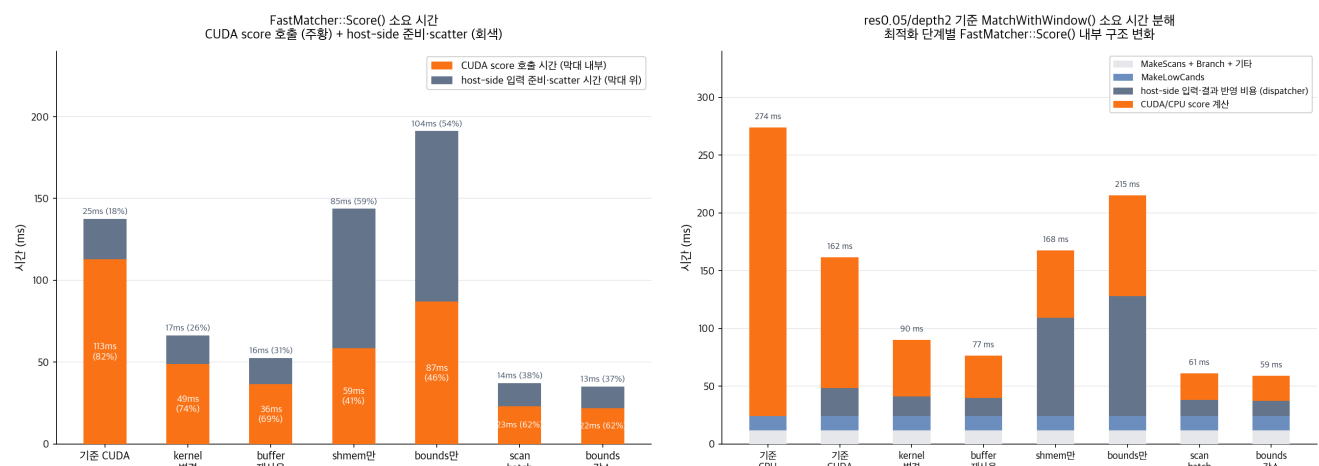


그림 5.2는 **FastMatcher::Score()** 실행 시간을 CUDA score 시간과 host dispatcher 시간으로 분해한 결과이다(res0.05/depth2, 60.3M 기준). CUDA score 시간은 H2D · kernel · D2H를 포함한 CUDA 호출 구간이며, host

dispatcher는 CUDA 호출 전후의 입력 배열 구성, score scatter, candidate 정렬 등 CPU 측 처리 시간이다. scan batch와 bounds 감소(batched 버전)는 CUDA score 시간이 전체 배치를 한 번에 처리하는 구간이다.

**host-side 호출 구조와 device-side kernel 효율이 서로 다른 개선을 버전 간 비교가 보여주었다.** kernel 변경은 CUDA score 시간을 113ms → 49ms로 줄였지만 dispatcher(17ms)는 그대로였다. buffer 재사용은 CUDA score 시간을 37ms로 더 낮추었고 dispatcher도 16ms로 유지되었다. 각 버전이 한 축씩 개선하는 구조에서, 두 요소가 서로 다른 개선 축으로 작용함을 확인할 수 있었다.

**shmem만·bounds만에서 dispatcher가 급증하는 이유.** shmem만과 bounds만은 각각 GPU kernel 시간을 58.6ms, 87.0ms로 줄이는 데 성공하였지만, 동시에 host-side dispatcher가 85ms, 104ms로 급증하였다. *기준 CUDA*의 dispatcher(24.6ms)와 비교하면 3.5x~4.2x 증가였다. per-scan 호출 구조가 유지된 상태에서 tile 또는 bounds 처리를 위한 추가 host-side input 구성 과정이 scan마다 반복되었고, 이 비용이 dispatcher 시간 증가로 나타난 것으로 해석되었다. 이 결과는 kernel만 개선해서는 전체 Score() 시간을 *기준 CUDA* 수준 이하로 낮출 수 없음을 보여주었다.

두 병목을 동시에 해결할 때 비로소 **Score()** 시간이 크게 줄었다. scan batch는 15회 호출을 1회로 통합하여 dispatcher를 14ms로, CUDA score를 23ms로 함께 줄였다. bounds 감소는 full\_inside candidate에 대해 bounds check와 2D 좌표 계산을 생략하여 CUDA score를 22ms로 추가 감소시켰다. MatchWithWindow() 전체 기준으로 기준 CPU(274ms) 대비 bounds 감소(59ms)가 4.6x 빠르게 나타났으며, 절대 시간 감소의 대부분은 Score() 내부에서 발생하였다.

### 5.3 bounds 감소 구현의 score 호출 내부 분석

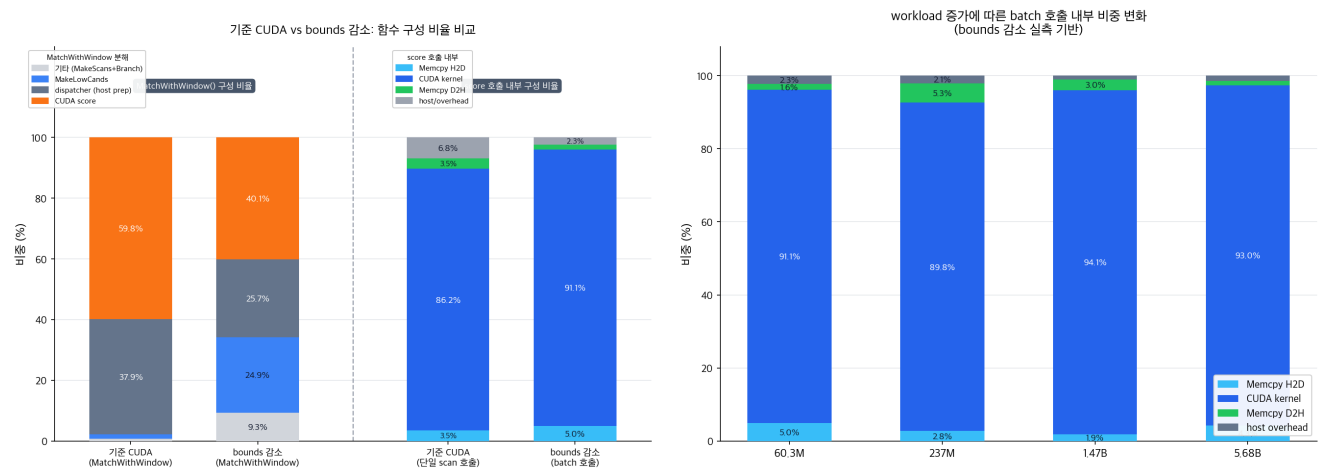


그림 5.3은 bounds 감소 구현의 단일 score 호출 내부를 H2D · kernel · D2H · host overhead로 분해한 결과이다. 좌측은 *기준 CUDA*와의 구성 비율 비교, 우측은 workload별 비중 변화를 나타낸다.

**batch 통합 이후에는 kernel이 병목의 중심이 되었다.** *기준 CUDA* 단일 scan 호출에서는 H2D(3.5%), D2H(3.5%), host overhead(6.8%)가 합산 13.8%를 차지하였다. 반면 bounds 감소 batch 호출에서는 kernel 비중이 91.1%, H2D+D2H는 6.6%에 그쳤다. 반복 overhead를 제거하자 Section 2.3에서 지적한 kernel 내부의 불규칙한 `grid[y*w+x]` read가 지배적 병목으로 남았다.

**workload 증가에 따라 kernel 비중이 더 높아졌다.** kernel 비중은 60.3M(91.1%) → 1.47B(94.1%)로 증가하였고 H2D+D2H 합산은 6.6% → 4.9%로 감소하였다. 연산량은 candidate × point 수에 비례하여 증가하지만 데이터 전송량은 candidate 수에만 비례하기 때문이다. 237M에서 H2D+D2H가 일시적으로 8.1%로 높아진 것은 candidate 수가 55,815 → 219,615(약 4x)로 늘어 D2H score 배열이 커진 영향이었다.

처리율의 한계는 알고리즘적 특성에서 비롯되었다. batch 구조 하에서도 처리율은 약 3.5 Gops/s에서 제한되었는데, candidate offset에 따라 불규칙하게 달라지는 `grid[y*w+x]` read 패턴이 coalescing을 막기 때문이다. grid cell 중복 제거(동일 cell을 참조하는 scan point를 한 번만 계산)를 시도한 결과 최대 78% 연산 감소가 가능하였으나, pose trajectory RMSE가 최대 0.05 m 증가하고 workload가 커질수록 효과가 줄어들어 최종 구현에는 포함하지 않았다.

---

## 6. 결론

본 과제는 Cartographer Fast Correlative Scan Matcher의 coarse scoring path를 대상으로, `FastMatcher::Score()`의 이중 병목—scan별 CUDA 반복 호출 overhead와 kernel 내부 비효율—을 동시에 해소하는 데 초점을 두었다.

**병목 분석.** profiling 결과, 기준 CUDA에서 `FastMatcher::Score()`가 `MatchWithWindow()` 전체 시간의 91.1%를 차지하며, 내부에는 서로 다른 두 개선 축이 존재하였다. host 측에서는 scan별로 반복되는 `cudaMalloc/cudaFree`와 dispatcher overhead가 누적되고, device 측에서는 candidate offset에 따라 불규칙하게 달라지는 `grid[y*w+x]` read로 global load efficiency(6.67%)와 warp execution efficiency(55.82%)가 낮았다. shmem만·bounds만 ablation 실험은 이 해석을 뒷받침한다. kernel을 개선하면서 per-scan 호출 구조를 그대로 두면 host-side dispatcher 비용이 오히려 급증하여 `Score()` 합계가 기준 CUDA보다 늦어진다. 한 축만 개선해서 는 전체 시간을 줄이는 데 한계가 있음을 실측으로 보여주었다.

**최적화 설계.** 두 병목을 함께 해소하기 위해 여러 scan의 point와 candidate를 하나의 연속 배열로 통합하는 batch 구조를 채택하였다. scan별 범위는 `point_offsets`, `cand_offsets` 메타데이터로 전달하고 kernel은 `blockIdx.y`로 scan을 구분하여 1회 launch로 전체 scan을 처리하였다. 이를 통해 15회 반복 호출을 1회로 줄였고, shared memory tiling·warp reduction으로 kernel 효율을 높였으며, full\_inside candidate에 대해 bounds check와 2D 좌표 계산을 1D cell offset으로 대체하였다.

**성능 결과.** 대표 조건 res0.05/depth2(60.3M)에서 bounds 감소 구현은 기준 CPU 대비 7.11x, 기준 CUDA 대비 3.9x의 speedup을 달성하였다. `FastMatcher::Score()` 시간 기준으로는 249.8 ms(기준 CPU) → 137.4 ms(기준 CUDA) → 35.1 ms(bounds 감소)로 감소하였다. 237M 조건에서는 최대 9.26x에 도달하며, CPU OpenMP SIMD(4 threads, 약 3.3x)의 2배 이상을 전 workload 구간에서 유지하였다. 단, bounds 감소는 1.47B 이상에서 precomputed 1D offset 배열 접근 증가로 scan batch보다 느려지는 한계가 있었으며, 두 구현 모두 91% 이상이 kernel 시간으로 구성되어 불규칙한 grid read 패턴이 다음 병목으로 남았다.